UNIVERSIDADE FEDERAL DO PARÁ
CAMPUS TUCURUÍ
FACULDADE DE ENGENHARIA

MIQUEIAS RODRIGUES RIBEIRO
LINDA INÉS RODRIGUES RIBEIRO
JESUS CLARINDO PINHEIRO

MANIPULAÇÃO DE AUTÔMATOS FINITOS E GRAMÁTICAS FORMAIS

TUCURUÍ – PA

2025

MIQUEIAS RODRIGUES RIBEIRO
LINDA INÉS RODRIGUES RIBEIRO
JESUS CLARINDO PINHEIRO

MANIPULAÇÃO DE AUTÔMATOS FINITOS E GRAMÁTICAS FORMAIS

Trabalho apresentado à disciplina de Teoria da Computação da Universidade Federal do Pará, como requisito parcial para avaliação do período 2025.2.
Professor: Otávio Noura Teixeira
TUCURUÍ – PA 2025

TUCURUÍ – PA

2025

1. INTRODUÇÃO

A Teoria da Computação estuda modelos formais de computação, como autômatos finitos e gramáticas formais, fundamentais para a compreensão das linguagens formais e compiladores.

Este trabalho tem como objetivo desenvolver um sistema capaz de:

- Criar e manipular autômatos finitos determinísticos (AFD) e não determinísticos (AFN)
- Converter AFN para AFD
- Minimizar AFD
- Criar e manipular Gramáticas Regulares (GR) e Gramáticas Livres de Contexto (GLC)
- Simplificar GLC passo a passo
- Converter GR para AFD
- Gerar árvore de derivação
- Gerar pseudocódigo do reconhecedor

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Autômato Finito Determinístico (AFD)

Um AFD é uma 5-tupla:

$(Q, \Sigma, \delta, q0, F)$

Onde:

- Q = conjunto de estados
- Σ = alfabeto
- δ = função de transição
- $q0$ = estado inicial
- F = conjunto de estados finais

O AFD aceita uma palavra se, após consumir todos os símbolos, terminar em um estado final.

2.2 Autômato Finito Não Determinístico (AFN)

O AFN é semelhante ao AFD, porém permite:

- Múltiplas transições para o mesmo símbolo
- Transições ε (vazias)

Foi implementada a conversão AFN → AFD pelo método dos subconjuntos.

2.3 Gramática Regular (GR)

Uma Gramática Regular possui produções do tipo:

A→aB
A → a

Ela é equivalente a um AFD.

Foi implementada:

- Criação de GR
- Conversão GR → AFD
- Exibição passo a passo das transições

2.4 Gramática Livre de Contexto (GLC)

Uma GLC possui produções do tipo:

A → γ

Onde γ pode conter terminais e não-terminais.

O sistema implementa:

- Identificação de problemas
- Processo completo de simplificação

3. FUNCIONALIDADES IMPLEMENTADAS

3.1 Manipulação de Autômatos

O sistema permite:

- Criar AFD
- Testar palavras
- Minimizar AFD
- Criar AFN
- Converter AFN para AFD
- Testar no AFD convertido

3.2 Manipulação de Gramáticas

O sistema permite:

- Definir não-terminais e terminais
- Inserir produções
- Converter GR → AFD
- Mostrar passo a passo da conversão
- Exibir transições no terminal

Exemplo exibido:

S --a--> A

S --b--> F

A --a--> F

4. SIMPLIFICAÇÃO DE GLC

Foi implementada uma opção de simplificação automática completa que executa:

1. Remoção de símbolos não geradores
2. Remoção de símbolos inalcançáveis
3. Remoção de produções ε
4. Remoção de produções unitárias
5. Verificação se a gramática é GLUD

O processo é exibido passo a passo no terminal.

Objetivo: aproximar a GLC ao máximo de uma Gramática Linear Unitária à Direita (GLUD).

5. DERIVAÇÃO DE PALAVRAS

O sistema implementa:

- Derivação recursiva
- Poda inteligente baseada em prefixo terminal
- Exibição da sequência de derivações

Exemplo:

$S \Rightarrow aA$

$aA \Rightarrow ab$

ab

Se a palavra for totalmente derivada, o sistema informa sucesso.

6. ÁRVORE DE DERIVAÇÃO

Foi implementada geração automática da árvore de derivação a partir do caminho de derivação.

Formato exibido:

```
  S
 / \
a   A
     |
     b
```

A árvore representa corretamente a aplicação das produções.

7. CONVERSÃO GR → AFD

A conversão é realizada analisando cada produção:

- A → aB gera transição δ(A,a) = B
- A → a gera transição para estado final artificial

O sistema mostra:

- Produção analisada
- Transição criada
- Estados finais
- Alfabeto
- Função de transição completa

8. GERAÇÃO DE PSEUDOCÓDIGO DO RECONHECEDOR

O sistema gera automaticamente o pseudocódigo equivalente ao autômato gerado pela gramática.

Exemplo gerado:

```
estado ← S

para cada símbolo c da palavra:

   se estado = S e c = 'a' então

      estado ← A

   se estado = S e c = 'b' então

      marcar como final

se estado é final então

   aceita

senão

   rejeita
```

Isso demonstra a equivalência formal entre gramática e autômato.

9. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento foram enfrentados problemas como:

- Tratamento incorreto de espaços na entrada
- Construção incorreta da árvore de derivação
- Conflito entre produções ao converter GR → AFD
- Identificação correta da produção utilizada na derivação

Todos os problemas foram resolvidos com:

- Normalização de entrada (strip)
- Reestruturação da construção da árvore
- Ajuste na lógica de transições
- Correção na análise da derivação

10. RESULTADOS

O sistema final:

- Implementa todos os requisitos do documento
- Funciona de forma interativa
- Exibe processos passo a passo
- Permite análise formal completa de linguagens regulares e livres de contexto

11. CONCLUSÃO

O projeto alcançou todos os objetivos propostos, integrando conceitos fundamentais da Teoria da Computação em uma aplicação funcional.

Foram implementados:

- Autômatos
- Conversões
- Minimização
- Gramáticas
- Simplificação
- Derivação

- Árvore de derivação
- Geração de pseudocódigo

O sistema demonstra, na prática, a equivalência entre autômatos finitos e gramáticas regulares, além de permitir a análise estrutural de gramáticas livres de contexto.

12. CONSIDERAÇÕES FINAIS

O desenvolvimento deste trabalho permitiu aprofundar a compreensão sobre:

- Linguagens formais
- Modelos computacionais
- Processos de reconhecimento
- Transformações formais

A implementação prática consolidou os conceitos teóricos estudados em sala.